

Software Quality Factors Analysis

The chosen application is a Kotlin mobile application. Its primary function is tracking daily medication that the user has to take. It can be connected to a smartwatch to receive sensor gathered data like heart rate. The application sends notifications when sensor values go above a threshold and when medication needs to be taken.

The Quality Model I've chosen is the **ISO 25010** from 2011.

Func. Suit.	Perform. Efficiency	Compatib.	Usability	Reliability	Security	Maintainability	Portability
Func. Compl.	Time behav	Co-existen.	Appropriat.	Maturity	Confid.	Modularity	Adaptab.
Func. Corect.	Resource util.	Interoperability	Learnability	Availability	Integrity	Reusability	Instability
Func. Appro.	Capacity		Operability	Fault tolerance	Non-repudiance	Analysability	Replaceability
			User error prot.	Recoverability	Authenticity	Modifiability	
			UI Asethet.		Accountab.	Testability	
			Accesibil.				

I've chosen 5 subfactors:

- **Resource Utilisation**
- **Recoverability**
- **Confidentiality**
- **Maintainability**
- **Installability**

Resource Utilization measurement:

- I will manually test on different physical devices using phones profiles (statistics collected by the phone to measure an apps performance)

Results: After two days of consistently using the application, I have found the following:

- the application used on average 95MB or memory, with a max usage of 277MB
- during a profiling session when trying out all functionalities, the CPU usage didn't go past 43%
- power: my phone battery has a capacity of 4200 mAh

Measure	MedNow	Reddit
Running duration	42 min 46 sec	1h 14 min 29 sec
Energy consumption	163.87 mAh	500 mAh
Energy Consumption Average	229.91 mA	402.56 mA
Battery life estimates	18.27 hours	10.44 hours

Conclusion: MedNow is more efficient power wise compared to a social media app.

Recoverability measurement:

- mean time to repair (average time it takes to restore the system after a failure) measured heuristically

Results:

- app starting after a phone restart: on average 62 seconds

65 seconds	55 seconds	55 seconds	68 seconds	67 seconds
------------	------------	------------	------------	------------

- app receiving information from the watch after it comes back in range: on average 7.44 seconds

7.84 sec	9.13 sec	5.27 sec	6.55 sec	8.96 sec	5.73 sec	9.01 sec	6.48 sec	7.19 sec	8.22 sec
----------	----------	----------	----------	----------	----------	----------	----------	----------	----------

Confidentiality measurement:

- static analysis by myself using a checklist (is there authentication, encrypted data transfer, etc.)

Result: 6/10 points (green = 1 point, yellow = 0.5 points, red and gray = 0 points)

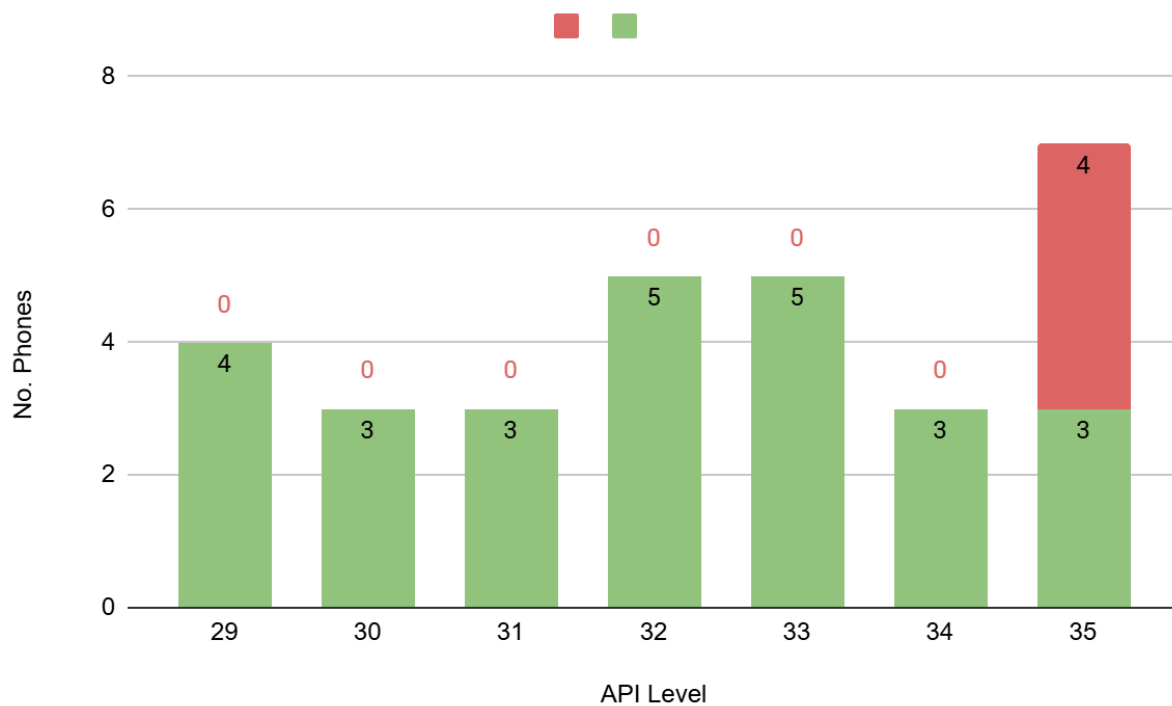
Question	Compliance
Is Firebase Authentication used?	Yes
Is the user's auth status always validated before database access?	No
Are tokens or user IDs hardcoded or logged?	Not hardcoded, logged once
Are database paths dynamically constructed with user ID?	Yes

Are all database references using HTTPS by default?	Yes
Are there any hardcoded API keys or tokens in the code?	No
Local Storage Confidentiality	Not applicable
Are there client-side security assumptions?	Yes
Are writes/reads scoped to authenticated users?	Yes
Is all the data transfer encrypted?	Yes between the app and the server, no between the app and smartwatch

Installability measurement:

- Manual installation on different devices (with different Android versions)
- And/Or the usage of Firebase test lab (emulated devices)

Results: Tested with 30 virtual and real phones, the results are below, green are passed tests, red failed tests



Maintainability measurement:

- Static analysis using automated tools in order to determine the Technical Debt and other issues.

Results: Using SonarQube i've identified the following:

- a total of 36 maintainability issues split as follows: 2 high, 22 medium, 11 low, 1 info
- the total technical debt in man-hours is 4 hours and 40 minutes
- the high issues were related to methods that had high cognitive complexity
- the medium issues were related to hardcoded version numbers and overall bad practices for kotlin development
- and the low issues were related to deprecated code

Conclusions:

- Sonarqube's rating is an A, it doesn't get better than that
- The fact that the debt to lines of code ratio is 0.4% is great, shows that there were minimal mistakes in the source code, and a little under 5 hours to fix all of them is basically less than a full work day, which is amazing for a project that took weeks in development.

